Convex and Non-Convex Cost Functions

Contents

1. Overview 2
2. The cost function is a function of the parameters 3
3. The chord test 4
   - 3.1 The definition as a formula 5
   - 3.2 Why convexity matters: one minimum 7
4. What goes wrong with a non-convex loss 8
5. Two losses compared 9
   - 5.1 A straight line: one bowl 9
   - 5.2 A tiny neural network: two minima and a ridge 10
6. Summary 13
7. Sources 13
8. Key terms 14

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Hessian and multivariate Taylor.
- **Leads to:** Convex sets and convex optimisation; Gradient descent.

1. Overview

Key point: A convex cost function is one bowl: every chord lies on or above the curve, any minimum is the global minimum, and gradient descent (with a small enough learning rate) finds it from any start. A **non-convex** (G-1333) one can have several dips, and gradient descent may stop in the wrong one.

The idea first appeared with **gradient descent** (G-862): a **convex** function (G-476) is one where a straight line between any two points on its curve never goes below the curve. Linear regression's loss is convex; other losses can trap **gradient descent** (G-862) in a **local minimum** (G-1110) instead of the **global minimum** (G-848).

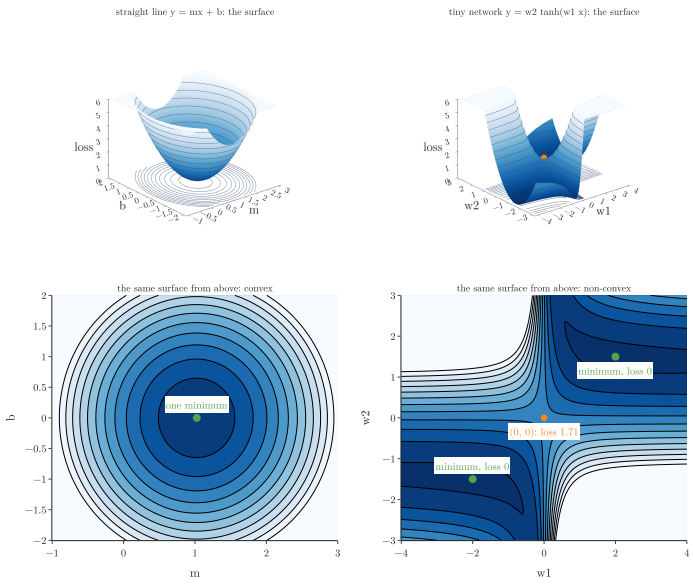


Figure 1: One dataset, two models. Top: the loss as a surface over the two parameters, with contour lines drawn on it and dropped to the floor. Bottom: the same surfaces seen from above, as contour maps. The loss of a straight line is one bowl (left); the loss of a tiny neural network has two separate minima with a ridge between them (right)

Figure 1 shows the difference on real losses. Both columns fit the same 21 points; only the model changes. For the straight line $y = mx + b$, the loss is a number for every pair (m, b),

written $L(m, b)$. For example, $L(1, 0) = 0.18$: the line $y = x$ misses the 21 points by a mean squared error (the average of the squared gaps) of 0.18.

The top row draws each loss as a surface: the two parameters run along the floor and the loss is the height above each pair. The line's surface is one bowl. The network's surface has two valleys with a ridge between them, which is why the orange point $(0, 0)$ sits higher, at loss 1.71.

The bottom row is the same surface seen from above, a **contour map** (see the partial derivatives and gradients): each line joins points of the same loss. The colours and the marked points are the same as on the surface above. Read it so: lines close together mean a steep slope, and a centre ring is the lowest point (green).

This Note writes the chord test as a formula, shows what it guarantees and what goes wrong without it, and checks both losses of Figure 1.

2. The cost function is a function of the parameters

> **Key point:** We fix the data and treat the loss as a function of the model's parameters; one parameter gives a curve, two give a surface, many give a surface we can no longer draw.

A **loss function** (G-706), or **cost function** (G-492), measures how far a model's predictions are from the true values (see the **regression metrics** (G-1652)). During training the data is fixed. What changes are the parameters, so we read the loss as a function of the parameters:

$$\text{loss} = L(\text{parameters})$$

Changing a parameter raises or lowers the loss. Training means finding the parameter values where the loss is lowest, so the loss acts as the guide that leads to the right model.

The number of parameters sets the shape we can draw:

- **One parameter**, such as the slope alone: the loss is a curve over a line.
- **Two parameters**, such as slope m and intercept b of simple linear regression: the loss is a surface over a plane, drawn as a contour map in Figure 1 (left).
- **Many parameters**, as in a neural network: the same idea in more dimensions, which we cannot draw.

Figure 2 shows the idea with one parameter. The 21 points never move. Only the slope m of the line $y = mx$ changes, from -1 to 3. Each slope gives the line its own errors (red sticks), and so its own loss: a point on the curve on the right. At $m = -1$ the loss is 6.18; at $m = 1.02$ it reaches its lowest value, 0.18.

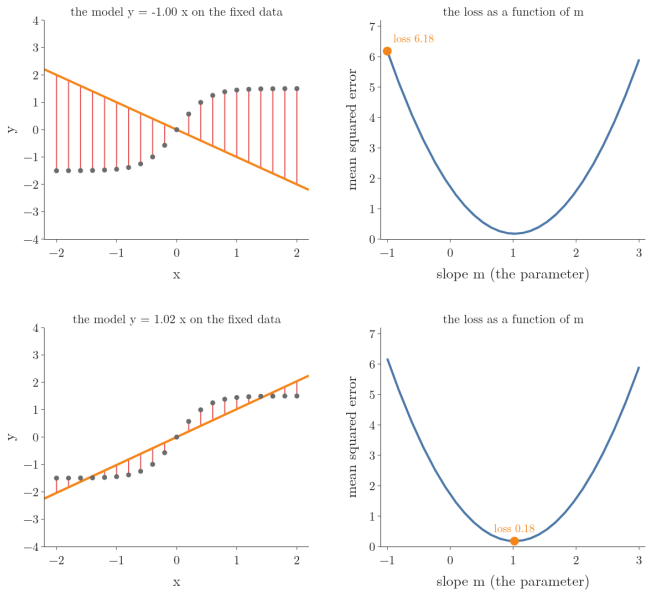


Figure 2: The loss as a function of one parameter. Left: the line $y = mx$ on the fixed data, with its errors, as the slope m changes from -1 to 3. Right: the mean squared error for each m; the lowest point is m = 1.02, loss 0.18.

The curve is built one point at a time. Each row below is one try: choose a slope, measure the errors of that line, and average their squares.

Slope m	Loss
-1	6.18
0	1.71
1	0.18
2	1.58
3	5.91

Plotting the five pairs gives five points of a U shape; trying every slope in between fills in the curve of Figure 2.

Whether this function is convex decides how easy its lowest point is to find.

3. The chord test

> **Key point:** Pick two points on the graph and join them with a straight line (a **chord**, G-384); the function is convex if, for every such pair, the curve between them never rises above the chord.

3.1 The definition as a formula

> **Key point:** A point a fraction of the way between a and b has a curve value no larger than the same fraction of the way between $f(a)$ and $f(b)$.

The picture "the chord lies on or above the curve" becomes a formula once we name the points between a and b. Every such point is $\theta a + (1-\theta)b$ for some θ between 0 and 1: $\theta = 1$ gives a, $\theta = 0$ gives b, $\theta = 0.5$ the midpoint. The chord's height above that point is the same mix of the two end heights.

1. **In words:** at every point between a and b, the curve is at or below the chord.
2. **Formula:** f is **convex** if, for all a, b and every θ with $0 \le \theta \le 1$,

$$f\Big(\theta a + (1-\theta)b\Big) \;\le\; \theta f(a) + (1-\theta) f(b)$$

   The left side is the curve; the right side is the chord.
3. **Example:** $f(w) = w^2$ with $a = -1$, $b = 3$ and $\theta = 0.5$. The midpoint is

$$0.5 \times (-1) + 0.5 \times 3 = 1$$

   The curve at the midpoint:

$$f(1) = 1$$

   The chord at the midpoint:

$$0.5 \times f(-1) + 0.5 \times f(3)$$

$$= 0.5 \times 1 + 0.5 \times 9 = 5$$

   $1 \le 5$: the curve is below the chord (Figure 3, left). The same holds for every pair of points, so w^2 is convex.

How to read Figure 3: each panel is the graph of one function, with the input w on the horizontal axis and the value on the vertical axis. The blue curve is the function. Pick two points on the horizontal axis, a and b; the black dots are the curve's values there. The orange straight line joining the dots is the chord. At the point halfway between a and b (dashed line), compare two heights: the green dot is the curve, the orange dot is the chord. If the green dot is below the orange dot (left), the curve dips under the chord. If the green dot is above (right), the curve bulges over the chord.

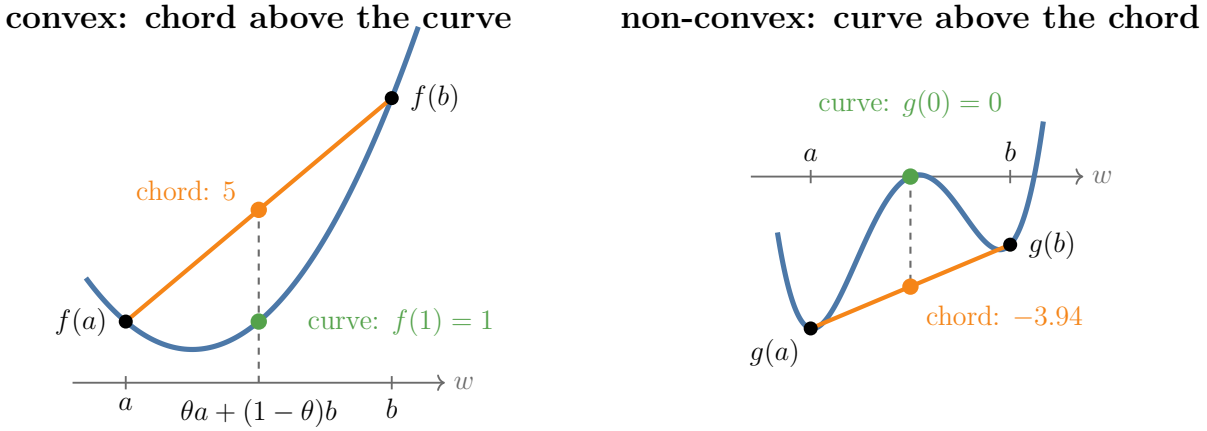


Figure 3: The chord test. Left: for w^2, the curve at the midpoint (1) is below the chord (5): convex. Right: for $g(w) = w^4 - 4w^2 + w$, the curve at the midpoint (0) is above the chord (-3.94): non-convex

A function that fails the test for even one pair of points is **non-convex** (G-1333). Take the function

$$g(w) = w^4 - 4w^2 + w$$

with $a = -1.5$, $b = 1.5$ and $\theta = 0.5$ (Figure 3, right). The curve at the midpoint:

$$g(0) = 0$$

The chord at the midpoint:

$$0.5 \times g(-1.5) + 0.5 \times g(1.5)$$

$$= 0.5 \times (-5.44) + 0.5 \times (-2.44)$$

$$= -3.94$$

Here $0 > -3.94$: the curve rises above the chord, so g is non-convex. One failing pair is enough.

> **Extra:** The same inequality with $<$ instead of $\leq$ (for $a \neq b$ and $0 < \theta < 1$) defines a **strictly convex** function (G-1899): the curve is strictly below every chord, never touching it in between. w^2 is strictly convex. The function $\max(0, |w| - 1)$ is convex but not strictly: it is flat at 0 between -1 and 1, and the chord from -0.5 to 0.5 lies exactly on the curve.

3.2 Why convexity matters: one minimum

> **Key point:** A strictly convex function has at most one minimum, and for any convex function every **local minimum** (G-1110) is a global minimum; a non-convex function can have several minima, and only one of them is the lowest.

Optimisation looks for the parameter values where the loss is lowest. Convexity gives two guarantees:

- **Any convex function:** every local minimum is a **global minimum** (G-848). There is no dip that is lower only locally.
- **Strictly convex:** there is at most one minimum point. A convex function that is not strict can have a flat bottom with many minimum points, but they all share the same lowest value.

The first guarantee follows from the chord test, in three steps:

1. Suppose some point c were a local minimum, and some other point d had a lower value, $f(d) < f(c)$.
2. The chord from c to d slopes downward, and convexity keeps the curve at or below the chord.
3. So just next to c the curve already dips below $f(c)$, and c was not a local minimum after all (Boyd and Vandenberghe §4.2.2).

The second guarantee follows the same way. Suppose a strictly convex f had two different minimum points c and d with the same lowest value f^*. The strict chord test at their midpoint gives

$$f\left(\frac{c+d}{2}\right) < \frac{1}{2}f^* + \frac{1}{2}f^*$$

$$\frac{1}{2}f^* + \frac{1}{2}f^* = f^*$$

so the midpoint has a value below the lowest one, which is impossible.

A **non-convex function** (G-1333) gives no such promise. In Figure 3 (right), g has a dip at $w = 1.35$ where $g = -2.62$ and a deeper one at $w = -1.47$ where $g = -5.44$. The first is a local minimum; the second is the global minimum.

> **Extra:** Points with zero slope are called **stationary points** (G-1879). For g, the derivative (the slope, **derivatives** (G-595)) is
>
> $$g'(w) = 4w^3 - 8w + 1$$
>
> It is zero at $w = -1.47$, 0.13 and 1.35. The second derivative (how the slope itself changes) tells them apart:

$$g''(w) = 12w^2 - 8$$

w	$g''(w)$	Shape	Kind
-1.47	18.0	curves up	global minimum
0.13	-7.8	curves down	maximum
1.35	13.8	curves up	local minimum

A zero slope alone does not say which kind of point we are at.

4. What goes wrong with a non-convex loss

Key point: Gradient descent only sees the local slope, so on a non-convex loss it settles in whichever dip it reaches first, which may be a **local minimum** (G-1110) with a worse loss than the global one.

Picture a ball placed on a hilly track. The ball rolls downhill and comes to rest at the bottom of whichever valley lies below its starting point. The ball cannot know that a deeper valley lies behind the next hill.

Gradient descent behaves like the ball. It steps downhill and stops where the slope is zero (see the **gradient descent** (G-862)). At a local minimum the slope is zero too, so the algorithm has no way to tell that a deeper dip exists elsewhere.

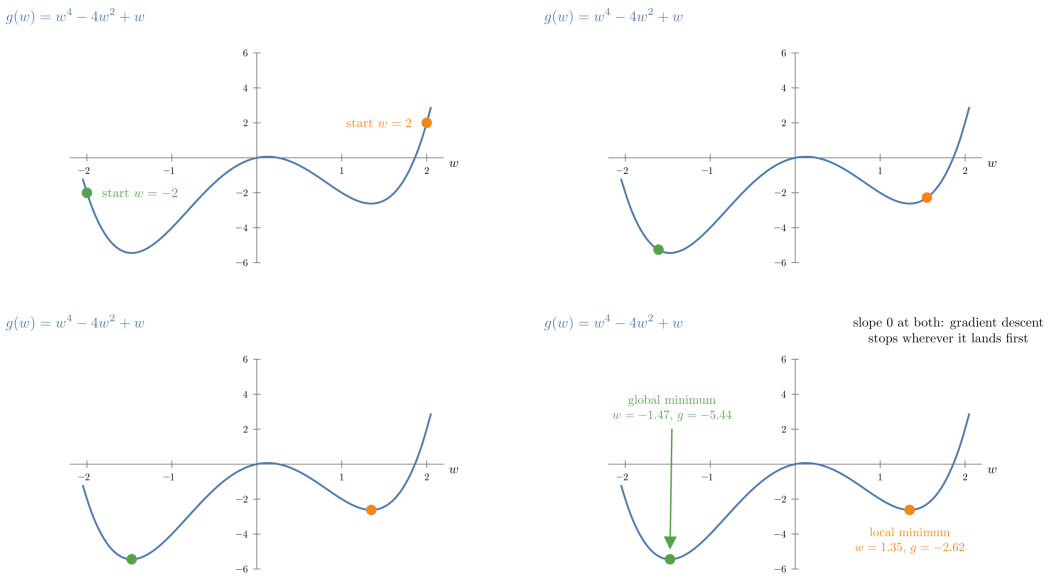


Figure 4: Gradient descent on $g(w) = w^4 - 4w^2 + w$ with learning rate 0.02, started at $w = -2$ (green) and $w = 2$ (orange). Key frames: the two starts, after two steps, after 30 steps, and the two end points

Figure 4 runs gradient descent on g from two starting points with the same learning rate:

Start	Ends at	Loss there	Kind
$w = -2$	$w = -1.47$	-5.44	global minimum
$w = 2$	$w = 1.35$	-2.62	local minimum

The orange run is the worst case for a **non-convex** (G-1333) loss: it converges, but to a sub-optimal answer. The parameters it returns give a loss 2.82 higher than the best possible. On a convex loss this cannot happen: every start ends at the same lowest value.

> **Extra:** Ways to reduce the risk: start from several random points and keep the best result, or use **stochastic gradient descent** (G-1892) (one observation per step), whose noisy steps can jump out of a shallow dip (Kleinberg et al. 2018). Methods built for this (momentum, adaptive learning rates) are covered with neural networks.

5. Two losses compared

> **Key point:** The loss of linear regression with squared error is convex, so it has one bowl; the loss of even a tiny neural network is **non-convex** (G-1333), with several minima and a ridge between them.

Figure 1 fits the same data with two models. The data is 21 points $x = -2, -1.8, \ldots, 2$ with $y = 1.5 \tanh(2x)$, an S-shaped curve (tanh is the S-shaped function that squeezes any number into the range -1 to 1; the tanh function). The loss is the mean squared error in both cases.

5.1 A straight line: one bowl

> **Key point:** Over its slope and intercept, the squared-error loss of a straight line is a single bowl with one lowest point.

For the line $y = mx + b$, the contours in Figure 1 (left) are closed rings around one point, $m = 1.02$, $b = 0$, with loss 0.18. That loss is not 0 because no straight line fits an S-curve exactly, but it is the best a line can do, and **gradient descent** (G-862) with a small enough learning rate reaches it from any start (Boyd and Vandenberghe §9.3).

The chord test confirms it. Take the two end points $(m, b) = (-1, 0)$ and $(3, 0)$. Their losses, 6.18 at $m = -1$ and 5.91 at $m = 3$, are rows of the table in section 2. The midpoint is $(1, 0)$:

$$\text{curve: } L(1, 0) = 0.18$$

$$\text{chord: } 0.5 \times 6.18 + 0.5 \times 5.91$$

$$= 3.09 + 2.955 = 6.05$$

In two dimensions the chord is a straight line through the parameter plane, and the surface stays below it. Figure 5 (left) walks along that straight line from $(-1, 0)$ to $(3, 0)$ and plots the loss on the way: a U-shaped curve that stays under the dashed chord everywhere. The curvature of the Hessian proves this holds for every linear regression loss: its Hessian never has a negative eigenvalue.

5.2 A tiny neural network: two minima and a ridge

> **Key point:** Flipping the signs of both weights gives the same network, so its loss has two equal minima; the straight line between them crosses a ridge, which breaks the chord test.

The smallest possible neural network, $y = w_2 \tanh(w_1 x)$, has one hidden unit (one tanh neuron) and two **weights** w_1 and w_2 (the numbers training adjusts). It can fit the data exactly: at $(w_1, w_2) = (2, 1.5)$ it reproduces $y = 1.5 \tanh(2x)$, with loss 0.

Because $\tanh(-z) = -\tanh(z)$, flipping both signs gives the same predictions: $(-1.5)\tanh(-2x) = 1.5\tanh(2x)$. So $(-2, -1.5)$ is a second **global minimum** (G-848), also with loss 0 (Figure 1, right).

The chord test between the two minima fails at the midpoint $(0, 0)$:

$$\text{curve: } L(0, 0) = 1.71$$

$$\text{chord: } 0.5 \times 0 + 0.5 \times 0 = 0$$

$1.71 > 0$, so the loss is **non-convex** (G-1333). A convex function could never have two separate lowest points with a higher point between them. Figure 5 (right) walks the straight path between the two minima: the loss climbs from 0 over a ridge of height 1.71 and comes back down to 0, entirely above the chord (red).

How to read Figure 5: a loss over two parameters cannot be drawn as one curve, so we walk along a straight path from one parameter pair to another and plot the loss on the way. The horizontal axis is the fraction of the way (0 at the start, 1 at the end, 0.5 at the midpoint), and the vertical axis is the loss. The result is a one-input curve, so the chord test of Section 3 applies to it: the dashed green line is the chord between the loss at the start and the loss at the end. A curve below the dashed line (left) passes; a curve above it (right, red) fails.

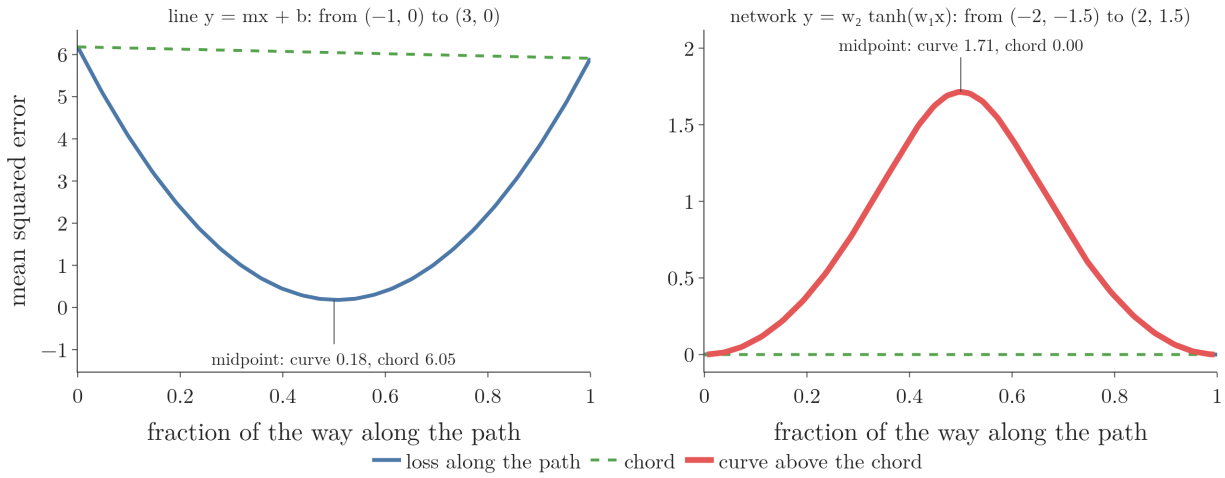


Figure 5: The chord test along a straight path in parameter space. Left: the line model, from $(m, b) = (-1, 0)$ to $(3, 0)$; the loss (blue) stays below the chord (dashed). Right: the tiny network, between its two minima $(-2, -1.5)$ and $(2, 1.5)$; the loss rises above the chord (red), to 1.71 at the midpoint.

Here both minima are equally good, so landing in either is fine. Real networks have millions of weights, many such symmetries, and also genuine local minima and flat stretches, so their losses are non-convex (Goodfellow et al. §8.2.2–8.2.3).

Extra: The midpoint $(0, 0)$ is itself a **stationary point** (G-1879): with $w_1 = 0$ every prediction is 0 whatever w_2 is, and with $w_2 = 0$ every prediction is 0 whatever w_1 is, so neither weight has a slope there. The midpoint is a **saddle point** (G-1718): the loss rises along one diagonal and falls along the other (see the Hessian). **Gradient descent** (G-862) started exactly at $(0, 0)$ would never move, since both slopes are zero there: one reason why neural network weights are not all started at zero (Goodfellow et al. §8.4).

Python: a numerical chord test. We pick 10,000 random pairs of parameter points and random θ, and record by how much the curve ever rises above the chord. A result above 0 proves the loss is non-convex; 0 is evidence (not proof) that it is convex.

```
import numpy as np

xs = np.linspace(-2, 2, 21)
ys = 1.5 * np.tanh(2 * xs)
line = lambda m, b: np.mean((ys - m * xs - b) ** 2)
net = lambda w1, w2: np.mean((ys - w2 * np.tanh(w1 * xs)) ** 2)

rng = np.random.default_rng(0)
def worst_gap(L, lo, hi):
    worst = 0
    for _ in range(10_000):
```

```
        p, q = rng.uniform(lo, hi), rng.uniform(lo, hi)
        t = rng.uniform()
        curve = L(*(t * p + (1 - t) * q))
        chord = t * L(*p) + (1 - t) * L(*q)
        worst = max(worst, curve - chord)
    return worst

worst_gap(line, [-1, -2], [3, 2])   # 0: never above the chord
worst_gap(net, [-4, -3], [4, 3])    # 6.42: non-convex
```

6. Summary

	Convex cost function	Non-convex cost function
Chord test	curve never above any chord	curve above some chord
Minima	every local minimum is global (one point if strictly convex)	can have several, only one lowest
Gradient descent	reaches the best value from any start	result depends on the start; may stop in a local minimum
Example	linear regression with squared error	neural networks

- The **gradient descent** (G-862) row follows from the minima row: gradient descent stops where the slope is zero, and on a convex loss every such low point is the global one.
- The loss is a function of the parameters; the data stays fixed, because training changes only the parameters, so the best model is the lowest point of this function.
- Convex: $f(\theta a + (1-\theta)b) \leq \theta f(a) + (1-\theta)f(b)$ for all a, b and $0 \leq \theta \leq 1$; this chord test guarantees that every **local minimum** (G-1110) is a global minimum.
- One failing pair of points is enough to show a function is **non-convex** (G-1333), because convexity must hold for every pair.
- On a non-convex loss, gradient descent can converge to a sub-optimal local minimum, because it only sees the local slope and stops wherever the slope is zero.
- So on a convex loss, such as linear regression's, gradient descent finds the best parameters from any start; on a non-convex loss, such as a neural network's, the start can decide where it ends.

7. Sources

Built from

- CampusX, "Difference between convex & non-convex cost function; what happens when cost function is non-convex?", YouTube, https://www.youtube.com/watch?v=TXVtbgaEyms
- Starmer, J. (StatQuest), "Gradient Descent, Step-by-Step", YouTube, https://www.youtube.com/watch
- Sanderson, G. (3Blue1Brown), "Gradient descent, how neural networks learn | Deep Learning Chapter 2", YouTube, https://www.youtube.com/watch?v=IHZwWFHWa-w

Other references

- Boyd, S. and Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press. Sections 4.2.2 (local and global optima) and 9.3 (gradient descent).
- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. Sections 8.2.2–8.2.3 (local minima, saddle points) and 8.4 (parameter initialisation).
- Kleinberg, R., Li, Y. and Yuan, Y. (2018). "An Alternative View: When Does SGD Escape Local Minima?". *ICML*.

8. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Non-convex function (G-1333)	A function whose curve is not one simple bowl: some straight line between two of its points (a chord) lies below part of the curve. It can have several local minima.
Cost function (G-492)	A measure of how far a model's predictions are from the true values, averaged over the data and read as a function of the model's parameters; training looks for the parameters that make it smallest. Often another name for the loss function.
Chord (G-384)	The straight line joining two points on a function's graph; it is used to test convexity: a convex function never rises above any of its chords.
Strictly convex function (G-1899)	A function whose curve lies strictly below the straight line (chord) joining any two different points of it; it has at most one minimum.
Stationary point (G-1879)	A point where a function is flat, its slope is zero (the derivative, or every partial derivative, is zero); it is a minimum, a maximum or a saddle point.
Gradient descent (G-862)	Finding the lowest point of a function by repeated small steps downhill.
Convex function (G-476)	A function where a straight line between any two points of its curve never goes below the curve; it has a single minimum.
Local minimum (G-1110)	A point lower than everything around it, but not the lowest overall.
Global minimum (G-848)	The lowest point of the whole function.
Loss function (error function) (G-706)	A formula that measures how wrong a model's predictions are; in deep learning, the error on one training row.
Regression metric (G-1652)	A number that summarises how close a regression model's predictions are to the true values.

Term	Meaning
Derivative (G-595)	The slope of a function at a point: how fast the output changes as the input changes there. It is zero at the lowest point of a smooth curve, which is how training finds a minimum.
Stochastic gradient descent (SGD) (G-1892)	Gradient descent that uses one random row for every update, so each update is cheap and many updates happen per epoch, at the cost of a noisier path.
Saddle point (G-1718)	A flat point that curves up in one direction and down in another.